Companion
to

ChatGPT Teaches TikZ

Chapter 29: Professional TikZ Workflow (A)

Contents

29.1 What This Companion Adds

Chapter 29 brought the advanced techniques together as a production method. This Companion develops a release workflow: establish sources of truth, isolate figures, debug reproducibly, separate correctness from optimization, inspect at publication size, and preserve everything needed for a clean build.

Design

A finished figure is not only one that looks correct today. It is one whose source, dependencies, generated artifacts, and verification procedure make the same result reproducible tomorrow.

29.2 A Production Vocabulary

Item or practice	Purpose
Shared style file	Holds project-wide visual conventions.
Figure source file	Holds the structure of one substantial picture.
Minimal example	Reproduces one failure without unrelated document code.
Clean build	Recreates output from declared sources and dependencies.
Revision test	Checks that ordinary changes remain local.
Visual inspection	Finds collisions, clipping, and poor final-size legibility.
Externalization cache	Speeds builds without becoming the source of truth.
Release archive	Preserves every source needed to reproduce the figures.

29.3 Decide the Sources of Truth

Project-wide appearance belongs in one shared file. Mathematical structure belongs in the figure source. Generated PDFs and externalization files are outputs, not authoritative definitions.

```
% tikzstyle.tex
\tikzset{
  math object/.style={draw,rounded corners,
    minimum width=12mm,minimum height=8mm},
  ordinary map/.style={-{Stealth},thick}
}
```

A figure should use these semantic names rather than duplicate their current options.

Common Mistake

If two files both define the same project-wide style, later edits can produce two incompatible truths. Give each shared decision one authoritative home.

29.4 Use a Predictable Project Layout

One practical structure is

```
book.tex
tikzstyle.tex
tikzlibrarymymath.code.tex
figures/
  comparison.tex
  construction.tex
external/
```

The source files are written by the author. The `external` directory is generated and can be rebuilt. A small inline picture need not be separated merely for symmetry; split a figure when independent testing or maintenance becomes easier.

29.5 Declare Dependencies Deliberately

```
\usetikzlibrary{
  arrows.meta,
  backgrounds,
  calc,
  fit,
  positioning
}
\input{tikzstyle.tex}
```

Do not depend on a library or style being loaded accidentally by another chapter. A clean, independent test of a figure should expose missing dependencies immediately.

29.6 Build in Controlled Passes

A reliable order is:

1. load libraries and shared definitions;
2. place principal named objects;
3. add structural edges;
4. add labels and exceptional geometry;
5. add regions, backgrounds, and decoration;
6. test revisions and inspect final-size output.

Compile after each pass. When an error appears, the most recent small group of changes is the natural search area.

29.7 Make Failures Reproducible

Reduce a failing figure to a complete small document that still exhibits the problem.

```
\documentclass{article}
\usepackage{tikz}
\usetikzlibrary{positioning}
\begin{document}
\begin{tikzpicture}
  \node[draw] (A) {A};
  \node[draw,right=of A] (B) {B};
  \draw[->] (A)--(B);
\end{tikzpicture}
\end{document}
```

Add the failing feature to this example, or remove features from the original until only the failure remains. The first reported error usually matters more than later cascading messages.

29.8 Debug Syntax and Geometry Separately

A syntax failure prevents compilation. Check braces, brackets, library names, node names, and the semicolon ending the previous path.

A geometric failure compiles but draws the wrong result. Mark important coordinates temporarily:

```
\coordinate (M) at ($(A)!0.5!(B)$);
\fill[red] (M) circle (1.5pt);
```

Remove the diagnostic mark after verifying the dependency. Do not hide an unexplained cause beneath an arbitrary shift.

29.9 Optimize Only a Correct Picture

Externalization can accelerate a large book, but cached output can conceal whether a changed source was rebuilt. First compile and inspect the figure normally. Then give stable figures stable externalized names and keep the cache in a generated directory.

```
\usetikzlibrary{external}
\tikzexternalize[prefix=external/]
\tikzsetnextfilename{comparison-diagram}
```

When diagnosing a visual discrepancy, rebuild the relevant picture or disable externalization temporarily. Performance machinery must never decide what the correct figure is.

29.10 Worked Project: A Maintainable Figure

29.10.1 Stage 1: Shared Definitions

```
\tikzset{
  math object/.style={draw,rounded corners,
    minimum width=15mm,minimum height=8mm},
  ordinary map/.style={-{Stealth},thick},
  map label/.style={fill=white,inner sep=1pt}
}
```

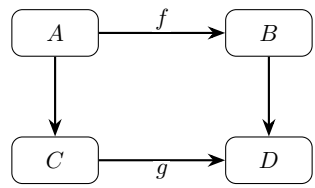
29.10.2 Stage 2: Figure Structure

```
\node[math object] (A) {$A$};
\node[math object,right=of A] (B) {$B$};
\node[math object,below=of A] (C) {$C$};
\node[math object,right=of C] (D) {$D$};
\draw[ordinary map] (A)--node[map label,above] {$f$} (B);
\draw[ordinary map] (A)--(C) (B)--(D) (C)--(D);
```

29.10.3 The Complete Picture

```
\begin{tikzpicture}[node distance=14mm and 22mm]
  \node[math object] (A) {$A$};
  \node[math object,right=of A] (B) {$B$};
  \node[math object,below=of A] (C) {$C$};
  \node[math object,right=of C] (D) {$D$};
  \draw[ordinary map]
    (A)--node[map label,above] {$f$} (B);
  \draw[ordinary map] (A)--(C);
  \draw[ordinary map] (B)--(D);
  \draw[ordinary map]
    (C)--node[map label,below] {$g$} (D);
\end{tikzpicture}
```

See



The release test lengthens one label, changes `node distance`, and restyles `ordinary map`. Each change should have one obvious source.

29.11 Inspect the Release Candidate

Check	Question
Legibility	Are labels readable at the final printed size?

Spacing	Do arrows, labels, nodes, and regions remain separate?
Consistency	Do equal mathematical roles look equal?
Typography	Does notation match the surrounding document?
Robustness	Do ordinary revisions remain local?
Reproducibility	Are all libraries, styles, figure files, and build requirements present?

Inspect rendered pages, not only isolated enlarged figures. Page placement can reveal crowding or a scale problem that is invisible during close editing.

29.12 Preserve a Rebuildable Release

Archive the main document, figure sources, shared styles, custom libraries, and any non-generated assets. Record special compilation requirements such as `-shell-escape`. Generated caches may be included for convenience, but they must not replace their sources.

Design

Test the archive by building from a clean directory. A successful build from the release materials is stronger evidence than a successful build in a working directory containing months of undeclared artifacts.

29.13 Debugging Workshop

29.13.1 A Figure Works Only Inside the Book

Its independent test is missing a library, style definition, macro, or package that the book loaded elsewhere.

29.13.2 The First Error Points at Innocent Code

Inspect the preceding path for an unclosed group or missing semicolon.

29.13.3 A Corrected Figure Still Looks Old

Rebuild or disable the externalization cache before changing geometry again.

29.13.4 The Archive Does Not Rebuild

Compare the clean directory with the working project and add the missing source or declared dependency.

29.14 Exercises

1. Identify one source of truth for each shared visual convention.
2. Design a directory layout for a book with several large figures.
3. Extract one substantial picture into its own source file.
4. Create a minimal complete example for one difficult feature.

5. Write separate syntax and geometry debugging procedures.
6. Perform three deliberate revision tests on one figure.
7. Inspect a figure at its intended publication size.
8. Assemble and test a clean release archive.

29.15 Solutions

29.15.1 Exercises 1–4

Keep shared design in one style file, individual structure in figure files, and generated output in a separate directory. A minimal example must include the document class, TikZ package, required libraries, and the smallest failing picture.

29.15.2 Exercises 5–8

Check syntax from the first error and geometry with temporary marks. Test a longer label, a spacing change, and a global style change. Inspect the rendered page, then rebuild the archived sources in a clean directory.

29.16 ChatGPT Workshop

Ask ChatGPT to support a controlled workflow without taking over design decisions.

1. Inventory this project's sources, generated files, and dependencies.
2. Reduce this TikZ failure to a complete minimal example.
3. Separate syntax diagnosis from geometric diagnosis.
4. Review the figure at publication size for collisions and inconsistency.
5. Propose a clean-build checklist and a revision-test checklist.

29.17 Final Checklist

Check that you can

1. give each shared decision one authoritative source;
2. organize figures and dependencies predictably;
3. build and compile in controlled passes;
4. reproduce failures in small complete examples;
5. distinguish correctness from build optimization;
6. inspect figures at publication size;
7. test ordinary revisions; and
8. preserve a clean, rebuildable release.